

Asynchronous Streams in C# with Examples

In this article, I am going to discuss **Asynchronous Streams in C#** with Examples. Please read our previous article where we discussed [How to Cancel a Non-Cancellable Task in C#](#) with Examples.

Asynchronous Streams in C#

From C# 8.0, we are not limited to returning a single value from an asynchronous method. We can return a sequence of values that are dynamically generated. In this article, first, we will talk about asynchronous streams. We will start this article by going over the `IEnumerable` interface and we will also see how to use the `yield` keyword. Then, we will create our first asynchronous stream using the `IAsyncEnumerable` interface. And, in the next article, we will see several ways to [cancel an asynchronous stream in C#](#).

Note: Asynchronous Streams is a new feature introduced in C# 8. So, in this article, I am using Visual Studio 2022 and targeting the .NET Core 3.1 to demonstrate the demos.

`IEnumerable` Interface and `yield` keyword in C#:

Before we talk about asynchronous streams, let's remember the use of the `IEnumerable` interface. The `IEnumerable` interface is the one that allows us to iterate on a type. For example, we can iterate over a list, it is because the list implements the `IEnumerable` interface. That means, if we have a list of strings, then we can iterate it in the following way.

```
//list of string
var names = new List<string>() { "Anurag", "Pranaya", "Sambit" };

//iterating over the list using foreach loop
foreach (var name in names)
{
    //You can do anything with the name
    //for example printing the name on the console
    Console.WriteLine(name);
}
```

We can iterate over a list. This is because the `List<T>` implements the `IEnumerable` interface. If you right-click on the list class and select go to definition, then you will see that the `List<T>` class implements the `IEnumerable<T>` interface as shown in the below image,

```

namespace System.Collections.Generic
{
    ...public class List<T> : IList<T>, ICollection<T>, IEnumerable<T>, IEnumerable, IList, ICollection,
    {
        ...public List();
        ...public List(int capacity);
        ...public List(IEnumerable<T> collection);
    }
}

```

As you can see in our example, we have a fixed list i.e. names (List<string>), which we can iterate over. In real life, this is really the case. It is likely that there will be a method that provides us with the elements of the list. Sometimes that method may return the full list or it could be a stream. And by a stream, I mean that it is going to return data over a period of time. Let us understand this with an example.

Let us create a method that is going to generate names over a period of time. And the question is how can we generate different values over a period of time on a method? Here, I am not talking about returning a fixed list which is very simple and straightforward. Here, I am talking about generating one value now, then another value in the future, and so on. Well, for this we can use the yield keyword in C#. With yield, we can define an iterator. Essentially what yield does is allow us to generate values one by one. The following method exactly does the same.

```

//This method is going to generate names over a period of time
private static IEnumerable<string> GenerateNames()
{
    yield return "Anurag";
    yield return "Pranaya";
    yield return "Sambit";
}

```

So, with this, we are making a stream in which first we are sending back the value Anurag and then after that, we're sending back the value Pranaya and then we are sending back the value Sambit. As the return type of this method is IEnumerable<string>. So, we can iterate the result of this GenerateNames method. For a better understanding, please have a look at the below image which iterates the results of the GenerateNames method.

```

foreach (var name in GenerateNames())
{
    //You can do anything with the name
    //for example printing the name on the console
    Console.WriteLine(name);
}

```

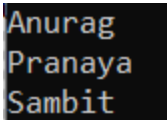
The complete example code is given below.

```

using System;
using System.Collections.Generic;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            ////list of string
            //var names = new List<string>() { "Anurag", "Pranaya", "Sambit" }
            ////iterating over the list using foreach loop
            //foreach (var name in names)
            //{
            //    //You can do anything with the name
            //    //for example printing the name on the console
            //    Console.WriteLine(name);
            //}
            foreach (var name in GenerateNames())
            {
                //You can do anything with the name
                //for example printing the name on the console
                Console.WriteLine(name);
            }
            Console.ReadKey();
        }
        //This method is going to generate names over a period of time
        private static IEnumerable<string> GenerateNames()
        {
            yield return "Anurag";
            yield return "Pranaya";
            yield return "Sambit";
        }
    }
}

```

Output:



```

Anurag
Pranaya
Sambit

```

When you run the above code, you will see the values Anurag, Pranaya, and Sambit on the console window. This is because our stream gives us those values.

Let us do an experiment. Let us return delay the method execution for 3 seconds before returning the last value from the GenerateNames method as shown in the below code.

```

using System;
using System.Collections.Generic;

```

```

using System.Threading;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            foreach (var name in GenerateNames())
            {
                //You can do anything with the name
                //for example printing the name on the console
                Console.WriteLine(name);
            }
            Console.ReadKey();
        }
        //This method is going to generate names over a period of time
        private static IEnumerable<string> GenerateNames()
        {
            yield return "Anurag";
            yield return "Pranaya";
            Thread.Sleep(3000);
            yield return "Sambit";
        }
    }
}

```

Output: Now run the above code and observe the output. The first and Second values you will get immediately. But after 3 seconds you will get the last value. So, this proves that our stream produces values over time.

How does Yield work in C#?

Now, let us understand how the yield is working. Please put a breakpoint on the foreach loop and you need to press the F11 key to debug the GenerateNames method.

First Iteration: When the foreach loop executes for the first time, it will invoke the GenerateNames method and it will return from the first yield statement and the value Anurag will print on the console window.

```

class Program
{
    static void Main(string[] args)
    {
        foreach (var name in GenerateNames())
        {
            Console.WriteLine(name);
        }

        Console.ReadKey();
    }

    private static IEnumerable<string> GenerateNames()
    {
        yield return "Anurag";
        yield return "Pranaya";
        Thread.Sleep(3000);
        yield return "Sambit";
    }
}

```

Iteration 1

When the foreach loop execute for the first time, it will invoke the GenerateNames method and it will return from the first yield statement and the value Anurag will print on the console window.

Second Iteration: When the foreach loop executes for the second time, it will not execute the first yield statement which is already executed by the previous iteration. So, it will start execution from where it is left. So, this time it will execute and return from the second yield statement and the value Pranaya will be printed on the console window.

```

class Program
{
    static void Main(string[] args)
    {
        foreach (var name in GenerateNames())
        {
            Console.WriteLine(name);
        }

        Console.ReadKey();
    }

    private static IEnumerable<string> GenerateNames()
    {
        yield return "Anurag";
        yield return "Pranaya";
        Thread.Sleep(3000);
        yield return "Sambit";
    }
}

```

Iteration 2:

When the foreach loop execute for the second time, it will not execute the first yield statement which is already executed by the previous iteration. So, it will start execution from where it is left. So, this time it will execute and return from the second yield statement and the value Pranaya will be printed on the console window.

Second Iteration: When foreach loop executes for the third time, it will not execute the first and second yield statements which are already executed by the previous iterations. So, it will start execution from where it is left. So, this time it will execute Thread.Sleep statement first which will delay the execution for 3 seconds and then it will execute the third yield statement and return the value Sambit which will be printed on the console window.

```

class Program
{
    static void Main(string[] args)
    {
        foreach (var name in GenerateNames())
        {
            Console.WriteLine(name);
        }

        Console.ReadKey();
    }

    private static IEnumerable<string> GenerateNames()
    {
        yield return "Anurag";
        yield return "Pranaya";
        Thread.Sleep(3000);
        yield return "Sambit";
    }
}

```

Iteration 3

When the foreach loop execute for the third time, it will not execute the first and second yield statements which are already executed by the previous iterations. So, it will start execution from where it is left. So, this time it will execute Thread.Sleep statement first which will delay the execution for 3 seconds and then it will execute the third yield statement and return the value Sambit which will be printed on the console window.

So, in this way, the Yield statement works in C#. So, actually, this is concurrency. I mean the GenerateNames method concurrently executing. So, what If I want to use Asynchronous Programming here? Let us see that.

Stream with Asynchronous Programming in C#:

For asynchronous programming, we need to do three changes as follows.

1. First, we need to use async in the method signature.
2. Second, we need to use Task or Task<T> as the return type.
3. Third, within the method body, somewhere we need to use await operator.

Let us do the above three in our GenerateNames method as follows:

```

1      2
private static async Task<IEnumerable<string>> GenerateNames()
{
    yield return "Anurag";
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3)); 3
    yield return "Sambit";
}

```

The following is the complete code.

```

using System;
using System.Collections.Generic;
using System.Threading;
using System.Threading.Tasks;

```

```

namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            foreach (var name in GenerateNames())
            {
                Console.WriteLine(name);
            }
            Console.ReadKey();
        }
        private static async Task<IEnumerable<string>> GenerateNames()
        {
            yield return "Anurag";
            yield return "Pranaya";
            await Task.Delay(TimeSpan.FromSeconds(3));
            yield return "Sambit";
        }
    }
}

```

With the above changes, you will see that, we will get the following compilation errors.

Code	Description	Project	File
CS1579	foreach statement cannot operate on variables of type 'Task<IEnumerable<string>>' because 'Task<IEnumerable<string>>' does not contain a public instance definition for 'GetEnumerator'	AsynchronousProgramming	Program.cs
CS1624	The body of 'Program.GenerateNames()' cannot be an iterator block because 'Task<IEnumerable<string>>' is not an iterator interface type	AsynchronousProgramming	Program.cs

1. The first compilation error saying that the **foreach statement cannot operate on variables of type 'Task<IEnumerable<string>>' because 'Task<IEnumerable<string>>' does not contain a public instance definition for 'GetEnumerator'.**
2. The second compilation error says that **the body of 'Program.GenerateNames()' cannot be an iterator block because 'Task<IEnumerable<string>>' is not an iterator interface type AsynchronousProgramming.**

This makes sense because we can iterate something that implements the IEnumerable interface. But if you go to the Task class, you will see that the Task class does not implement the IEnumerable as shown in the below image.

```

namespace System.Threading.Tasks
{
    ... public class Task : IThreadPoolWorkItem, IAsyncResult, IDisposable
    {
        ...
    }
}

```

Task does not Implement the IEnumerable interface

So, therefore we cannot iterate over a Task and therefore we are getting some compilation errors. But what if we have some sort of stream in which we want to do asynchronous operations?

Asynchronous Steam Operations in C#:

We can use asynchronous steams to create IEnumerable that generates data asynchronously. For this, we can use the IAsyncEnumerable interface. As its name implies IAsyncEnumerable is the asynchronous version of IEnumerable. Therefore, it allows us to perform iterations where the operations are asynchronous.

First, modify the GenerateNames method as shown in the below image. Here, instead of Task<IEnumerable<string>>, we are using IAsyncEnumerable<string> as the return type. With this change, you will not get any compile-time error in the GenerateNames method.

```
private static async IAsyncEnumerable<string> GenerateNames()
{
    yield return "Anurag";
    yield return "Pranaya";
    await Task.Delay(TimeSpan.FromSeconds(3));
    yield return "Sambit";
}
```

The second change that we need to do is we need to use await for each loop as shown in the below image. Some people are confused by adding the await operator just before the function name and that is wrong. We just need to add await before the for each loop.

```
await foreach(var name in GenerateNames())
{
    Console.WriteLine(name);
}
```

The above for each loop is created inside the Main method. As we are using await operator inside the Main method, we need to use the async Main method. The complete code is given below.

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static async Task Main(string[] args)
        {
            await foreach(var name in GenerateNames())
            {
                Console.WriteLine(name);
            }
            Console.ReadKey();
        }
    }
}
```

```

    }
    private static async IEnumerable<string> GenerateNames()
    {
        yield return "Anurag";
        yield return "Pranaya";
        await Task.Delay(TimeSpan.FromSeconds(3));
        yield return "Sambit";
    }
}

```

Output: You will get the same output as the previous example using IEnumerable

```

Anurag
Pranaya
Sambit

```

And the most important thing is that we are not blocking the thread, just like we did in the previous example where we use Thread.Sleep. Here, we are using an asynchronous operation, which means that we are not blocking any threads. The asynchronous streams are maybe useful when you have to get information out of a Web service that has pagination in it and you have to iterate over the different pages of the Web service and you can use Yield to return the different batches of the information of the Web service just so that you don't have to keep all of the information in memory, but that you can process it as soon as you have it on your application.

In the next article, I am going to discuss the [Canceling of Asynchronous Streams in C#](#) with Examples. Here, in this article, I try to explain **Asynchronous Streams in C#** with Examples. I hope you enjoy this Asynchronous Streams in C# with Examples article.